W10 HARD

SCENARIO PRACTICE

MQTT + SMQTT + CoAP

Long questions, hard patterns, traps, and final revision

WHAT THIS PDF TRAINS

- Long 8-mark scenario answers
- MQTT architecture and topic logic
- Wildcard + and # reasoning
- SMQTT secure broadcast flow
- CoAP protocol selection and troubleshooting
- CON/NON/ACK/RST decision questions
- MQTT vs CoAP integrated designs
- Rapid short questions + exam strategy

Top Priority

MQTT Architecture -> Topics/Wildcards -> SMQTT -> CoAP -> Message Types

Then: CoAP vs HTTP -> MQTT vs CoAP -> Status Codes -> Integrated Design

Answer pattern: Identify -> Reason -> 5-7 points -> Conclusion

Scenario 1: Smart Home Temperature Monitoring

Long-question pattern - identify, justify, explain

SCENARIO

A smart home has several temperature sensors. Each sensor sends readings to a central system. Multiple mobile applications should automatically receive the updates without talking directly to each sensor.

QUESTION

Which protocol and architecture best fit this requirement? Explain the roles of the main components.

Best answer: MQTT Publish-Subscribe

- Use MQTT because it is a lightweight publish-subscribe protocol used with TCP/IP.
- Temperature sensors act as publishers.
- The central message broker receives published data and dispatches it.
- Mobile applications act as subscribers.
- Each published message carries a topic that guides broker routing.
- Subscribers receive messages whose topics match their subscriptions.
- Publishers and subscribers do not need to know each other directly, which supports scalable communication.

MEMORY

P -> B -> S = Publisher -> Broker -> Subscriber

Scenario 2: Low-Bandwidth Remote Sensors

Long-question pattern - identify, justify, explain

SCENARIO

A company installs small embedded sensors in remote locations. Bandwidth is limited and the device software must remain very small.

QUESTION

Which W10 protocol is designed for this environment? Give the exact design reasons.

Best answer: MQTT

- MQTT is designed for remote connections.
- It is suitable for limited-bandwidth links.
- It has a small-code footprint.
- It is lightweight and intended for embedded connectivity.
- It uses publish-subscribe communication with a message broker.
- It works in conjunction with TCP/IP.

MEMORY

RLS = Remote - Limited bandwidth - Small code

Scenario 3: Identify MQTT Components

Long-question pattern - identify, justify, explain

SCENARIO

A soil-moisture sensor sends readings, a central service receives and distributes those readings, and a monitoring app wants to receive only soil-moisture messages.

QUESTION

Identify the publisher, broker, and subscriber, and explain each role.

Best answer: Sensor = Publisher, Service = Broker, App = Subscriber

- Publisher: produces and publishes data, such as sensor readings.
- Broker: connects publishers and subscribers and classifies sensor data into topics.
- Subscriber: an application interested in receiving data for a subscribed topic.
- The broker is the central communication point for dispatching messages.
- The components communicate through the topic rather than by direct publisher-to-subscriber addressing.

MEMORY

PSB = Publisher - Subscriber - Broker

Scenario 4: Topic-Based Routing

Long-question pattern - identify, justify, explain

SCENARIO

A living-room temperature sensor publishes under house/living-room/temperature. A mobile app wants only that data.

QUESTION

Explain how MQTT uses the topic to route the message.

Best answer: Topic controls broker routing

- An MQTT topic is a string used for routing.
- Topic hierarchy levels are separated by a slash (/).
- The publisher includes the topic in the published message.
- The broker reads the topic and finds matching subscribers.
- A client can subscribe to the exact topic.
- The broker delivers matching messages to that subscriber.

MEMORY

Topic = routing information for the broker

Scenario 5: Single-Level Wildcard

Long-question pattern - identify, justify, explain

SCENARIO

Available topics include house/living-room/temperature, house/kitchen/temperature, house/bedroom/temperature, and house/kitchen/humidity. The app wants all rooms' temperature readings only.

QUESTION

Which MQTT subscription should be used, and why?

Best answer: house/+/temperature

- Use house/+/temperature.
- The plus sign (+) is a single-level wildcard.
- It allows an arbitrary value at one hierarchy level.
- It matches living-room, kitchen, bedroom, or another single room level.
- It still requires temperature at the final hierarchy level, so humidity does not match.

MEMORY

+ = one hierarchy level

Scenario 6: Whole-House Wildcard

Long-question pattern - identify, justify, explain

SCENARIO

A dashboard must receive every message under the house topic, including temperature, humidity, lighting, security, and deeper subtopics.

QUESTION

Which wildcard should be used? Explain the difference from +.

Best answer: house/#

- Use house/#.
- The hash symbol (#) is a multilevel wildcard.
- It subscribes to all underlying hierarchy levels.
- It can include deeper subtrees beneath house.
- In contrast, + matches only one hierarchy level.

MEMORY

Plus = One Place | Hash = Whole House

Scenario 7: Secure Broadcast Messaging

Long-question pattern - identify, justify, explain

SCENARIO

A smart building controller must encrypt one command once and deliver it to multiple authorized IoT nodes.

QUESTION

Which W10 protocol extension fits this requirement? Explain the security idea and stages.

Best answer: SMQTT

- SMQTT is Secure MQTT, proposed to enhance MQTT security.
- It uses lightweight attribute-based encryption.
- Its main advantage is broadcast encryption.
- One encrypted message can be delivered to multiple nodes.
- The four main stages are setup, encryption, publish, and decryption.
- Publishers and subscribers register with the broker during setup.
- Subscribers with the appropriate master secret key decrypt the published data.

MEMORY

SEPD = Setup - Encryption - Publish - Decryption

Scenario 8: Constrained REST Device

Long-question pattern - identify, justify, explain

SCENARIO

A tiny battery-powered IoT node has little memory and bandwidth. The developer wants HTTP-like REST operations such as reading, updating, and deleting resources.

QUESTION

Which protocol should be used? Explain why.

Best answer: CoAP

- CoAP is a specialized web transfer protocol for constrained nodes and networks.
- It is an application-layer protocol.
- It follows a request-response model.
- It provides familiar REST-style methods such as GET, POST, PUT, and DELETE.
- It is designed for low memory, low bandwidth, low power, and high packet-failure environments.
- It can carry payloads such as XML, JSON, and CBOR.
- Its core protocol specification is RFC 7252.

MEMORY

CoAP = HTTP REST for small IoT devices

Scenario 9: CoAP vs HTTP Selection

Long-question pattern - identify, justify, explain

SCENARIO

System A is a normal web application. System B is a tiny IoT device with tight memory, power, and bandwidth limits.

QUESTION

Which system is better suited to CoAP? Compare the key protocol differences that justify the choice.

Best answer: System B -> CoAP

- CoAP is designed for constrained IoT environments.
- CoAP runs over UDP, while HTTP typically uses TCP.
- CoAP uses compact binary headers rather than heavier HTTP-style text headers.
- CoAP reduces the available options and method set to simplify implementation.
- CoAP keeps REST-like request-response behavior and familiar methods.
- CoAP is designed to use minimal resources on both device and network.

MEMORY

CoAP = small + UDP + compact | HTTP = general web + usually TCP

Scenario 10: Thermostat Request-Response

Long-question pattern - identify, justify, explain

SCENARIO

A phone requests bedroom temperature using coap://myhome.in:5683/nest_bedroom/temp and the thermostat returns the value.

QUESTION

Explain the CoAP request-response idea and identify the important URI parts.

Best answer: CoAP Request-Response

- The client sends a request to a CoAP server resource.
- coap:// identifies the protocol.
- myhome.in identifies the host.
- 5683 is the default CoAP port shown in the slide.
- nest_bedroom identifies the device/resource area.
- temp identifies the requested parameter/resource.
- The server returns the requested temperature value.

MEMORY

CoAP port = 5683

Scenario 11: Critical Alarm Delivery

Long-question pattern - identify, justify, explain

SCENARIO

A fire alarm message is critical. The sender must know it was received; if acknowledgement does not arrive, the sender should retransmit.

QUESTION

Which CoAP message type should be used? Explain the behavior.

Best answer: CON - Confirmable

- A confirmable (CON) message requires a response.
- The response may be a positive or negative acknowledgement.
- If acknowledgement is not received, retransmission occurs.
- Retransmissions continue until the allowed attempts are exhausted.
- This is suitable when delivery confirmation matters.

MEMORY

Critical + must confirm = CON

Scenario 12: Periodic Sensor Update

Long-question pattern - identify, justify, explain

SCENARIO

A temperature sensor sends a new measurement every few seconds. Missing one reading has little impact because another arrives shortly.

QUESTION

Which CoAP message type is more suitable? Why?

Best answer: NON - Non-confirmable

- A non-confirmable (NON) message does not normally require acknowledgement.
- The slide gives periodic sensor measurements as an example.
- It is suitable when occasional message loss has limited impact.
- It avoids the acknowledgement exchange required by confirmable messages.

MEMORY

Frequent data + some loss acceptable = NON

Scenario 13: ACK or RST?

Long-question pattern - identify, justify, explain

SCENARIO

A receiver successfully processes one confirmable CoAP message, but another received message contains data that cannot be parsed.

QUESTION

Which CoAP responses are appropriate in the two cases?

Best answer: ACK for success, RST for failure

- ACK is sent to acknowledge a confirmable (CON) message.
- RST represents Reset and acts as a negative acknowledgement.
- RST generally indicates a failure such as being unable to parse received data.
- These are two of the four CoAP message types together with CON and NON.

MEMORY

CNAR = CON - NON - ACK - RST

Scenario 14: CoAP Method Selection

Long-question pattern - identify, justify, explain

SCENARIO

A smart-home app must read a temperature, send new data, update an actuator setting, delete a resource, and observe continuing changes.

QUESTION

Choose the CoAP methods for each task.

Best answer: GET, POST, PUT, DELETE, OBSERVE

- GET is used to obtain a resource value.
- POST is used for sending/creating data in the REST-style interface.
- PUT is used to update a resource.
- DELETE removes a resource.
- OBSERVE is listed in the slides as a CoAP feature not present in HTTP.
- CoAP method calls can use confirmable or non-confirmable message services.

MEMORY

GPPDO = GET - POST - PUT - DELETE - OBSERVE

Scenario 15: Status-Code Interpretation

Long-question pattern - identify, justify, explain

SCENARIO

A CoAP client receives 2.05, 4.04, and 5.03 from three different requests.

QUESTION

Classify the three responses and state the slide meanings.

Best answer: 2.05 Content, 4.04 Not Found, 5.03 Service Unavailable

- 2.xx codes indicate successful outcomes; 2.05 means Content.
- 4.xx codes indicate client errors; 4.04 means Not Found.
- 5.xx codes indicate server errors; 5.03 means Service Unavailable.
- The status-code pattern is similar in spirit to grouped web response categories, but the CoAP set is reduced.

MEMORY

2 = success | 4 = client error | 5 = server error

Scenario 16: MQTT or CoAP?

Long-question pattern - identify, justify, explain

SCENARIO

System A must push sensor events to many dashboards automatically. System B must let a phone directly request and modify a thermostat resource.

QUESTION

Choose MQTT or CoAP for each system and justify the decision.

Best answer: A = MQTT, B = CoAP

- MQTT uses publish-subscribe communication and a broker, so it fits event distribution to many subscribers.
- MQTT routes messages using topics.
- CoAP uses request-response communication and REST-style methods.
- CoAP fits direct resource access using URIs and methods such as GET/PUT.
- The protocols are both lightweight but use different communication models.

MEMORY

Events / broker / topics -> MQTT | REST / URI / request-response -> CoAP

Scenario 17: Smart Hospital Full Design

Long-question pattern - identify, justify, explain

SCENARIO

A hospital needs wearable telemetry to reach multiple dashboards, secure broadcast messages for authorized nodes, direct actuator control, critical commands that require confirmation, and periodic non-critical readings.

QUESTION

Design a W10 solution using MQTT, SMQTT, and CoAP.

Best answer: Integrated MQTT + SMQTT + CoAP

- Use MQTT for telemetry because publish-subscribe allows one publisher to reach multiple subscribers through a broker.
- Use MQTT topics to organize different telemetry streams.
- Use SMQTT where the scenario requires broadcast encryption and enhanced MQTT security.
- Use CoAP for direct REST-style actuator/resource access.
- Use CON for critical CoAP messages requiring acknowledgement and retransmission.
- Use NON for periodic readings where an occasional missing value has little impact.
- The design uses each protocol according to the behavior described in the W10 slides.

MEMORY

Telemetry -> MQTT | Secure broadcast -> SMQTT | Direct REST control -> CoAP

Hard Trap Questions

True/False + one-line correction

1. MQTT uses request-response as its main communication model.

Ans: FALSE - MQTT uses publish-subscribe.

2. MQTT publishers must know subscriber addresses.

Ans: FALSE - the broker and topics decouple publishers and subscribers.

3. The + wildcard can match several hierarchy levels.

Ans: FALSE - + is single level.

4. The # wildcard can cover an entire lower subtree.

Ans: TRUE.

5. SMQTT is proposed to enhance MQTT security.

Ans: TRUE.

6. CoAP mainly runs over TCP.

Ans: FALSE - the slides describe CoAP over UDP.

7. CoAP supports REST-style GET, POST, PUT, and DELETE.

Ans: TRUE.

8. NON messages always require an acknowledgement.

Ans: FALSE - NON normally does not return ACK.

9. ACK acknowledges a confirmable message.

Ans: TRUE.

10. RST is a positive acknowledgement.

Ans: FALSE - it represents Reset / negative acknowledgement.

11. OBSERVE is listed as new in CoAP and not present in HTTP.

Ans: TRUE.

12. The slide shows 5683 as the default CoAP port.

Ans: TRUE.

Rapid Practice - 30 Short Questions

Try without looking at the notes

1. MQTT full form?
2. Which communication model does MQTT use?
3. Which protocol suite is MQTT used with?
4. Who introduced MQTT?
5. In which year was MQTT introduced?
6. When was MQTT standardized by OASIS?
7. What is a publisher?
8. What is a subscriber?
9. What is a broker?
10. What is an MQTT topic?
11. What separates topic hierarchy levels?
12. What does + wildcard mean?
13. What does # wildcard mean?
14. Name the five MQTT methods.
15. What is SMQTT?
16. What encryption idea is used by SMQTT?
17. What is broadcast encryption?
18. Name the four SMQTT stages.
19. CoAP full form?
20. What type of environment is CoAP designed for?
21. CoAP mainly uses TCP or UDP?
22. What default CoAP port appears in the slide?
23. Name four basic CoAP methods.
24. What is OBSERVE?
25. What is CON?
26. What is NON?
27. What is ACK?
28. What is RST?
29. What do 2.xx / 4.xx / 5.xx mean?
30. Main difference between MQTT and CoAP?

Final Exam Strategy

How to identify unseen W10 scenarios quickly

MASTER DECISION MAP

Broker + topic + many subscribers	->	MQTT
One-level wildcard	->	+
Whole lower subtree	->	#
Secure MQTT broadcast encryption	->	SMQTT
REST + UDP + URI	->	CoAP
Critical message needs ACK	->	CON
Periodic data, loss acceptable	->	NON
Positive acknowledgement	->	ACK
Negative/reset response	->	RST
2.xx / 4.xx / 5.xx	->	Success / Client / Server

8-MARK ANSWER FORMULA

- 1. Identify the protocol / feature immediately.
- 2. Write a one-line definition.
- 3. Connect scenario clues to the exact slide terms.
- 4. Write 5-7 clean points in order.
- 5. Add one exact method, wildcard, port, or message type.
- 6. End with a short conclusion.

ULTIMATE MEMORY CHAIN

PSB -> CDSUP -> +/# -> SEPD -> UDP/5683 -> GPPDO -> CNAR -> 2/4/5